# 🛡️ Elite Phishing URL Detection System - Complete Architecture

## 📋 Table of Contents


1. System Overview
2. Project Structure
3. Component Architecture
4. Data Flow
5. Feature Engineering Pipeline
6. TGIS (Trust Graph Intelligence System)
7. Machine Learning Pipeline
8. API Integrations
9. REST API Specification
10. Database Schema
11. Deployment Architecture
12. Implementation Checklist


---

## 🎯 System Overview

### Purpose

A production-grade phishing URL detection system combining traditional ML, external security APIs, and novel graph-based trust analysis.

### Key Capabilities

- **Real-time URL classification** (phishing/safe)
- **50+ engineered features** from URL structure, domain metadata, and content
- **External validation** via Google Safe Browsing & WHOIS
- **Graph-based anomaly detection** using Trust Graph Intelligence System (TGIS)
- **Ensemble ML models** (Random Forest + XGBoost + TGIS)
- **REST API** for integration
- **Interactive dashboard** for analysis

### Tech Stack

```
Backend:       Python 3.10+, FastAPI, Uvicorn
ML:            scikit-learn, XGBoost, NetworkX
APIs:          Google Safe Browsing, WHOIS, DNS
Database:      PostgreSQL, Redis (caching)
Frontend:      Streamlit
Deployment:    Docker, Docker Compose
Monitoring:    Prometheus, Grafana (optional)
```

## Project Structure

```
phishing-url-detector/
├── .env.example                    # Environment variables template
├── .gitignore
├── README.md
├── ARCHITECTURE.md                 # This file
├── requirements.txt                # Python dependencies
├── docker-compose.yml              # Docker orchestration
├── Dockerfile                      # Container image
│
├── config/
│   ├── config.yaml                 # Application configuration
│   ├── logging.yaml                # Logging configuration
│   └── model_params.yaml           # ML hyperparameters
│
├── data/
│   ├── raw/                        # Original datasets
│   │   ├── phishing_urls.csv
│   │   ├── legitimate_urls.csv
│   │   └── README.md               # Data sources documentation
│   ├── processed/                  # Processed features
│   │   ├── features.parquet
│   │   ├── train.parquet
│   │   ├── test.parquet
│   │   └── validation.parquet
│   ├── models/                     # Trained model artifacts
│   │   ├── random_forest_v1.pkl
│   │   ├── xgboost_v1.pkl
│   │   ├── scaler.pkl
│   │   ├── feature_names.json
│   │   └── metadata.json           # Model versioning info
│   ├── graphs/                     # Trust graph data
│   │   ├── domain_graph.gpickle
│   │   └── trust_scores.json
│   └── cache/                      # API response cache
│
├── src/
│   ├── __init__.py
│   │
│   ├── core/
│   │   ├── __init__.py
│   │   ├── config.py               # Configuration loader
│   │   ├── logger.py               # Logging setup
│   │   └── exceptions.py           # Custom exceptions
│   │
│   ├── data/
│   │   ├── __init__.py
│   │   ├── loader.py               # Dataset loading utilities
│   │   ├── preprocessor.py         # Data cleaning
│   │   └── splitter.py             # Train/test split logic
```

```
│   │
│   ├── features/
│   │   ├── __init__.py
│   │   ├── base.py                 # Abstract feature extractor
│   │   ├── url_features.py         # URL structure features (15)
│   │   ├── domain_features.py      # Domain metadata features (20)
│   │   ├── content_features.py     # Page content features (15)
│   │   ├── graph_features.py       # TGIS graph features (10)
│   │   └── pipeline.py             # Feature extraction pipeline
│   │
│   ├── external/
│   │   ├── __init__.py
│   │   ├── safe_browsing.py        # Google Safe Browsing API
│   │   ├── whois_client.py         # WHOIS lookup client
│   │   ├── dns_resolver.py         # DNS queries
│   │   ├── ssl_checker.py          # SSL certificate validation
│   │   └── cache_manager.py        # API response caching
│   │
│   ├── graph/
│   │   ├── __init__.py
│   │   ├── builder.py              # Graph construction
│   │   ├── analyzer.py             # Graph-based scoring
│   │   ├── trust_propagation.py    # Trust score calculation
│   │   └── anomaly_detection.py    # Graph anomaly detection
│   │
│   ├── models/
│   │   ├── __init__.py
│   │   ├── base_model.py           # Abstract model class
│   │   ├── random_forest.py        # Random Forest trainer
│   │   ├── xgboost_model.py        # XGBoost trainer
│   │   ├── ensemble.py             # Model ensemble logic
│   │   ├── trainer.py              # Training orchestration
│   │   └── evaluator.py            # Model evaluation metrics
│   │
│   ├── prediction/
│   │   ├── __init__.py
│   │   ├── predictor.py            # Main prediction service
│   │   ├── explainer.py            # SHAP-based explanations
│   │   └── batch_predictor.py      # Batch processing
│   │
│   └── utils/
│       ├── __init__.py
│       ├── validators.py           # URL/input validation
│       ├── metrics.py              # Custom metrics
│       └── helpers.py              # Utility functions
│
├── api/
│   ├── __init__.py
│   ├── main.py                     # FastAPI application
│   ├── dependencies.py             # Dependency injection
│   ├── middleware.py               # Custom middleware
│   │
│   ├── routes/
│   │   ├── __init__.py
```

```
│   │   ├── predict.py              # Prediction endpoints
│   │   ├── analyze.py              # Analysis endpoints
│   │   ├── health.py               # Health check endpoints
│   │   └── admin.py                # Admin endpoints
│   │
│   ├── schemas/
│   │   ├── __init__.py
│   │   ├── request.py              # Request models
│   │   ├── response.py             # Response models
│   │   └── internal.py             # Internal data models
│   │
│   └── services/
│       ├── __init__.py
│       ├── prediction_service.py   # Business logic
│       └── analysis_service.py     # Analysis logic
│
├── dashboard/
│   ├── __init__.py
│   ├── app.py                      # Main Streamlit app
│   ├── components/
│   │   ├── __init__.py
│   │   ├── predictor.py            # Prediction UI
│   │   ├── visualizer.py           # Visualization components
│   │   ├── graph_viewer.py         # Graph visualization
│   │   └── metrics.py              # Metrics dashboard
│   └── utils/
│       ├── __init__.py
│       └── helpers.py              # Dashboard utilities
│
├── notebooks/
│   ├── 01_data_exploration.ipynb           # EDA
│   ├── 02_feature_engineering.ipynb        # Feature analysis
│   ├── 03_model_training.ipynb             # Model experiments
│   ├── 04_graph_analysis.ipynb             # TGIS development
│   ├── 05_model_evaluation.ipynb           # Performance analysis
│   └── 06_deployment_testing.ipynb         # Integration tests
│
├── tests/
│   ├── __init__.py
│   ├── conftest.py                 # Pytest fixtures
│   ├── unit/
│   │   ├── test_url_features.py
│   │   ├── test_domain_features.py
│   │   ├── test_graph_builder.py
│   │   └── test_models.py
│   ├── integration/
│   │   ├── test_api_endpoints.py
│   │   ├── test_pipeline.py
│   │   └── test_external_apis.py
│   └── performance/
│       └── test_inference_speed.py
│
├── scripts/
│   ├── download_data.py            # Download datasets
```

```
│   ├── train_models.py             # Train all models
│   ├── build_graph.py              # Build trust graph
│   ├── evaluate_models.py          # Evaluate performance
│   └── deploy.sh                   # Deployment script
│
└── docs/
    ├── API.md                      # API documentation
    ├── FEATURES.md                 # Feature documentation
    ├── TGIS.md                     # TGIS algorithm details
    └── DEPLOYMENT.md               # Deployment guide
```

---

## 🏗 Component Architecture

### 1. Feature Engineering Layer

```
┌─────────────────────────────────────────────────────────────┐
│                    Feature Engineering Pipeline              │
├──────────────────┬──────────────────┬──────────────────────┤
│  URL Features    │ Domain Features  │   Content Features   │
│  (15 features)   │  (20 features)   │   (15 features)      │
├──────────────────┴──────────────────┴──────────────────────┤
│              External API Integration Layer                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │
│  │ Safe Browse  │  │    WHOIS     │  │     DNS      │      │
│  └──────────────┘  └──────────────┘  └──────────────┘      │
└─────────────────────────────────────────────────────────────┘
                              ↓
                     [60 Combined Features]
```

### 2. TGIS (Trust Graph Intelligence System)

```
┌─────────────────────────────────────────────────────────────┐
│                     Trust Graph Layer                        │
│                                                              │
│  Nodes: URL, Domain, IP, Registrar, NameServer, SSL Issuer  │
│                                                              │
│  ┌──────────┐    ┌──────────┐    ┌──────────┐              │
│  │  Graph   │──▶│  Trust   │──▶│ Anomaly  │              │
│  │ Builder  │    │  Scorer  │    │ Detector │              │
│  └──────────┘    └──────────┘    └──────────┘              │
│                                                              │
│  Output: 10 graph-based features + trust score              │
└─────────────────────────────────────────────────────────────┘
```

### 3. ML Model Layer

```
┌──────────────────────────────────────────────────────────────┐
│                    Machine Learning Models                   │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │Random Forest │  │   XGBoost    │  │     TGIS     │       │
│  │  (weight:    │  │  (weight:    │  │  (weight:    │       │
│  │    0.4)      │  │    0.4)      │  │    0.2)      │       │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘       │
│         └─────────────────┴─────────────────┘               │
│                            ↓                                 │
│                   [Ensemble Predictor]                       │
│                                                              │
│  Final Score = 0.4*RF + 0.4*XGB + 0.2*TGIS                   │
└──────────────────────────────────────────────────────────────┘
```

### 4. Service Layer

```
┌──────────────────────────────────────────────────────────────┐
│                      REST API (FastAPI)                      │
│                                                              │
│  /predict          - Single URL prediction                   │
│  /batch-predict    - Bulk URL analysis                       │
│  /analyze/{id}     - Detailed analysis report                │
│  /explain          - SHAP-based explanation                  │
│  /health           - System health check                     │
│  /metrics          - Model performance metrics               │
└──────────────────────────────────────────────────────────────┘
```

---

## 🔄 Data Flow

### End-to-End Prediction Flow

```
[User Input: URL]
      ↓
[Input Validation]
      ↓
[Check Cache] ──────────────────────────┐
      ↓                                 │
[Feature Extraction Pipeline]           │
      ├─ URL Structure Features         │
      ├─ Domain WHOIS Lookup            │
      ├─ DNS Resolution                 │
      ├─ SSL Certificate Check          │
      ├─ Safe Browsing API              │
      └─ Content Analysis (optional)    │
      ↓                                 │
[Graph Feature Extraction]              │
```

```
        ├─ Query Trust Graph                  │
        ├─ Calculate Trust Scores             │
        └─ Detect Anomalies                   │
        ↓                                     │
  [Feature Vector: 70 dimensions]            │
        ↓                                     │
  [Model Inference]                          │
        ├─ Random Forest → P1                 │
        ├─ XGBoost → P2                       │
        └─ TGIS → P3                          │
        ↓                                     │
  [Ensemble: 0.4*P1 + 0.4*P2 + 0.2*P3]    │
        ↓                                     │
  [Post-processing & Explanation]            │
        ↓                                     │
  [Cache Result] ←──────────────────────────┘
        ↓
  [Return Prediction + Metadata]
```

---

## 🧩 Feature Engineering Pipeline

### 1. URL Structure Features (15 features)

**File:** `src/features/url_features.py`

```python
class URLFeatureExtractor:
    """Extract features from URL string structure."""

    def extract(self, url: str) -> dict:
        """
        Features:
        1. url_length: int
        2. domain_length: int
        3. path_length: int
        4. num_dots: int
        5. num_hyphens: int
        6. num_underscores: int
        7. num_slashes: int
        8. num_question_marks: int
        9. num_equals: int
        10. num_at_symbols: int
        11. num_ampersands: int
        12. num_special_chars: int (total)
        13. has_ip_address: bool (0/1)
        14. has_port: bool (0/1)
        15. subdomain_count: int
        """
        return features
```

**Implementation Details:**

- Parse URL using `urllib.parse`
- Use regex for IP address detection
- Count special characters: `!@#$%^&*()+=[]{}|;:,<>?`
- Extract subdomains by splitting on `.`

---

### 2. Domain Features (20 features)

**File:** `src/features/domain_features.py`

```
class DomainFeatureExtractor:
    """Extract features from domain metadata."""

    def extract(self, domain: str) -> dict:
        """
        Features from WHOIS:
        1. domain_age_days: int (days since creation)
        2. domain_expiry_days: int (days until expiry)
        3. domain_registration_length: int (total registration period)
        4. is_registered: bool (0/1)
        5. registrar_reputation_score: float (0-1)

        Features from DNS:
        6. dns_record_count: int
        7. has_mx_record: bool (0/1)
        8. has_spf_record: bool (0/1)
        9. num_nameservers: int

        Features from SSL:
        10. ssl_certificate_valid: bool (0/1)
        11. ssl_certificate_age_days: int
        12. ssl_issuer_trusted: bool (0/1)

        Features from reputation APIs:
        13. alexa_rank: int (or -1 if unavailable)
        14. google_indexed: bool (0/1)
        15. page_rank_score: float (0-10, estimated)

        Features from domain string analysis:
        16. domain_in_brand_list: bool (0/1) - Check against top brands
        17. tld_suspicious: bool (0/1) - .tk, .ml, .ga, etc.
        18. shortest_word_length: int
        19. longest_word_length: int
        20. domain_entropy: float (Shannon entropy)
        """
        return features
```

**Implementation Details:**

- WHOIS lookup via `python-whois` library
- DNS queries via `dnspython`
- SSL check via `ssl` and `OpenSSL` modules
- Maintain brand list: top 10K domains from Alexa/Tranco
- TLD suspicious list: `.tk`, `.ml`, `.ga`, `.cf`, `.gq`, `.xyz`, `.top`
- Calculate Shannon entropy: `H(X) = -Σ p(x) log p(x)`

---

### 3. Content Features (15 features)

**File:** `src/features/content_features.py`

```
class ContentFeatureExtractor:
    """Extract features from page content (when accessible)."""

    def extract(self, url: str, html_content: str = None) -> dict:
        """
        Features (set to -1 if content unavailable):
        1. has_login_form: bool (0/1)
        2. num_external_links: int
        3. num_internal_links: int
        4. external_internal_ratio: float
        5. has_iframe: bool (0/1)
        6. num_redirects: int
        7. favicon_matches_domain: bool (0/1)
        8. has_popup: bool (0/1) - Check for window.open in JS
        9. uses_javascript_obfuscation: bool (0/1)
        10. html_title_brand_mismatch: bool (0/1)
        11. num_images: int
        12. num_forms: int
        13. form_has_password_field: bool (0/1)
        14. uses_https: bool (0/1)
        15. has_mixed_content: bool (0/1) - HTTPS page with HTTP resources
        """
        return features
```

**Implementation Details:**

- Fetch HTML with `requests` (timeout: 5s)
- Parse with `BeautifulSoup`
- Use `requests.head()` for redirect count
- Check favicon domain match
- Detect obfuscation: eval, unescape, fromCharCode patterns
- Compare title text with known brand names

---

### 4. Graph Features (10 features)

**File:** `src/features/graph_features.py`

```python
class GraphFeatureExtractor:
    """Extract features from trust graph."""

    def extract(self, url: str, graph: nx.Graph) -> dict:
        """
        Features:
        1. domain_cluster_size: int
        2. suspicious_neighbor_count: int
        3. cluster_phishing_ratio: float (0-1)
        4. registrar_trust_score: float (0-1)
        5. ip_shared_domains_count: int
        6. nameserver_trust_score: float (0-1)
        7. ssl_issuer_trust_score: float (0-1)
        8. graph_centrality_score: float (0-1) - Betweenness centrality
        9. community_detection_label: int
        10. anomaly_score_in_cluster: float (0-1)
        """
        return features
```

---

## 🕸 TGIS (Trust Graph Intelligence System)

### Graph Schema

**File:** `src/graph/builder.py`

```python
# Node Types
NODE_TYPES = {
    'URL': {
        'attributes': ['url', 'label', 'first_seen', 'last_seen']
    },
    'DOMAIN': {
        'attributes': ['domain', 'tld', 'creation_date', 'trust_score']
    },
    'IP': {
        'attributes': ['ip_address', 'country', 'asn', 'trust_score']
    },
    'REGISTRAR': {
        'attributes': ['name', 'reputation_score']
    },
    'NAMESERVER': {
        'attributes': ['hostname', 'trust_score']
    },
    'SSL_ISSUER': {
        'attributes': ['issuer_name', 'trust_level']
    }
}

# Edge Types
EDGE_TYPES = {
```

```
    'URL_TO_DOMAIN': {'weight': 1.0},
    'DOMAIN_TO_IP': {'weight': 0.8},
    'DOMAIN_TO_REGISTRAR': {'weight': 0.7},
    'DOMAIN_TO_NAMESERVER': {'weight': 0.6},
    'DOMAIN_TO_SSL_ISSUER': {'weight': 0.5},
    'URL_TO_URL': {'weight': 0.9, 'type': 'redirect'},
    'DOMAIN_TO_DOMAIN': {'weight': 0.4, 'type': 'shares_ip'}
}
```

### Trust Propagation Algorithm

**File:** `src/graph/trust_propagation.py`

```
def calculate_trust_score(graph: nx.Graph, node_id: str) -> float:
    """
    Trust Propagation Algorithm:

    1. Initialize trust scores:
       - Known legitimate nodes: 1.0
       - Known phishing nodes: 0.0
       - Unknown nodes: 0.5

    2. Iterative propagation (10 iterations):
       For each node:
           new_trust = (1-α) * current_trust + α * weighted_avg(neighbor_trust)
           where α = 0.3 (damping factor)

    3. Normalize scores to [0, 1]

    Returns: float (0 = suspicious, 1 = trusted)
    """
    pass
```

### Anomaly Detection

**File:** `src/graph/anomaly_detection.py`

```
def detect_anomalies(graph: nx.Graph, node_id: str) -> float:
    """
    Graph-based anomaly detection:

    1. Extract ego network (2-hop neighbors)
    2. Calculate graph metrics:
       - Degree centrality
       - Clustering coefficient
       - PageRank
    3. Compare with cluster statistics
    4. Use Isolation Forest on graph features
```

```
    Returns: anomaly_score (0 = normal, 1 = anomalous)
    """
    pass
```

### Community Detection

**File:** `src/graph/analyzer.py`

```
def detect_communities(graph: nx.Graph) -> dict:
    """
    Louvain algorithm for community detection.

    Returns: {node_id: community_id, ...}
    """
    from community import community_louvain
    return community_louvain.best_partition(graph)
```

---

## 🤖 Machine Learning Pipeline

### Model Training Flow

**File:** `src/models/trainer.py`

```
class ModelTrainer:
    """Orchestrates model training pipeline."""

    def train(self):
        """
        Training Pipeline:

        1. Load processed features
        2. Split data (70% train, 15% val, 15% test)
        3. Handle class imbalance:
           - SMOTE for oversampling minority class
           - Class weights in models
        4. Train Random Forest:
           - 5-fold cross-validation
           - Hyperparameter tuning with GridSearchCV
        5. Train XGBoost:
           - Early stopping on validation set
           - Feature importance analysis
        6. Save models and metadata
        7. Evaluate on test set
        """
        pass
```

### Random Forest Configuration

**File:** `src/models/random_forest.py`

```
RF_PARAMS = {
    'n_estimators': 500,
    'max_depth': 20,
    'min_samples_split': 10,
    'min_samples_leaf': 5,
    'max_features': 'sqrt',
    'class_weight': 'balanced',
    'random_state': 42,
    'n_jobs': -1,
    'oob_score': True
}
```

### XGBoost Configuration

**File:** `src/models/xgboost_model.py`

```
XGB_PARAMS = {
    'n_estimators': 300,
    'max_depth': 8,
    'learning_rate': 0.1,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'min_child_weight': 3,
    'gamma': 0.1,
    'reg_alpha': 0.1,
    'reg_lambda': 1.0,
    'scale_pos_weight': 1.5,  # Adjust based on class ratio
    'random_state': 42,
    'tree_method': 'hist',
    'eval_metric': 'logloss'
}
```

### Ensemble Strategy

**File:** `src/models/ensemble.py`

```
class EnsemblePredictor:
    """Combines predictions from multiple models."""

    def predict(self, features: np.ndarray) -> dict:
        """
        Ensemble Logic:

        1. Get predictions from each model:
           - rf_proba = random_forest.predict_proba(features)
           - xgb_proba = xgboost.predict_proba(features)
```

```
            - tgis_score = graph_analyzer.get_trust_score(url)

        2. Combine with weighted average:
           final_score = 0.4 * rf_proba[1] +
                         0.4 * xgb_proba[1] +
                         0.2 * (1 - tgis_score)

        3. Apply threshold (default: 0.5):
           prediction = 'phishing' if final_score > 0.5 else 'safe'

        Returns: {
            'prediction': str,
            'confidence': float,
            'rf_score': float,
            'xgb_score': float,
            'tgis_score': float,
            'final_score': float
        }
        """
        pass
```

---

## 🔌 API Integrations

### 1. Google Safe Browsing API

**File:** `src/external/safe_browsing.py`

```
class SafeBrowsingClient:
    """Google Safe Browsing API v4 integration."""

    def __init__(self, api_key: str):
        self.api_key = api_key
        self.base_url = "https://safebrowsing.googleapis.com/v4"

    def check_url(self, url: str) -> dict:
        """
        Check if URL is flagged.

        Endpoint: POST /threatMatches:find

        Request:
        {
            "client": {"clientId": "phishing-detector", "clientVersion": "1.0"},
            "threatInfo": {
                "threatTypes": ["MALWARE", "SOCIAL_ENGINEERING"],
                "platformTypes": ["ANY_PLATFORM"],
                "threatEntryTypes": ["URL"],
                "threatEntries": [{"url": "http://example.com"}]
            }
        }
```

```
        Returns:
        {
            'is_threat': bool,
            'threat_types': list,
            'platform_type': str,
            'threat_entry_type': str,
            'cache_duration': str
        }

        Rate Limit: 10,000 requests/day (free tier)
        Cache responses for 24 hours
        """
        pass
```

**Setup:**

1. Get API key: https://console.cloud.google.com/apis/credentials
2. Enable Safe Browsing API
3. Add to `.env`: `SAFE_BROWSING_API_KEY=your_key`

---

### 2. WHOIS Lookup

**File:** `src/external/whois_client.py`

```
class WHOISClient:
    """WHOIS data lookup with caching."""

    def lookup(self, domain: str) -> dict:
        """
        Query WHOIS data.

        Library: python-whois

        Returns:
        {
            'domain_name': str,
            'creation_date': datetime,
            'expiration_date': datetime,
            'updated_date': datetime,
            'registrar': str,
            'registrant_country': str,
            'name_servers': list,
            'status': list,
            'dnssec': str
        }

        Error Handling:
        - Return None if domain not found
        - Cache results for 7 days
```

```
        - Retry on network errors (3 attempts)
        """
        import whois
        try:
            w = whois.whois(domain)
            return self._parse_whois(w)
        except Exception as e:
            logger.error(f"WHOIS lookup failed for {domain}: {e}")
            return None
```

---

### 3. DNS Resolution

**File:** `src/external/dns_resolver.py`

```
class DNSResolver:
    """DNS record lookup."""

    def resolve(self, domain: str) -> dict:
        """
        Query DNS records.

        Library: dnspython

        Returns:
        {
            'A_records': list,      # IPv4 addresses
            'AAAA_records': list,   # IPv6 addresses
            'MX_records': list,     # Mail servers
            'NS_records': list,     # Name servers
            'TXT_records': list,    # Text records (SPF, DMARC)
            'CNAME_records': list,  # Canonical names
            'SOA_record': dict      # Start of Authority
        }
        """
        import dns.resolver
        pass
```

---

### 4. SSL Certificate Check

**File:** `src/external/ssl_checker.py`

```
class SSLChecker:
    """SSL certificate validation."""

    def check(self, domain: str) -> dict:
        """
        Validate SSL certificate.
```

```
        Returns:
        {
            'is_valid': bool,
            'issuer': str,
            'subject': str,
            'not_before': datetime,
            'not_after': datetime,
            'serial_number': str,
            'version': int,
            'is_trusted': bool,
            'chain_length': int
        }
        """
        import ssl
        import socket
        pass
```

---

### 5. Caching Strategy

**File:** `src/external/cache_manager.py`

```
class CacheManager:
    """Redis-based API response caching."""

    CACHE_DURATIONS = {
        'safe_browsing': 86400,    # 24 hours
        'whois': 604800,            # 7 days
        'dns': 3600,                # 1 hour
        'ssl': 86400                # 24 hours
    }

    def get(self, key: str, source: str) -> dict:
        """Retrieve from cache."""
        pass

    def set(self, key: str, value: dict, source: str):
        """Store in cache with TTL."""
        pass
```

---

## 🌐 REST API Specification

### API Endpoints

**File:** `api/main.py`

```python
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(
    title="Phishing URL Detector API",
    version="1.0.0",
    description="Elite phishing detection with ML + TGIS"
)

# Middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"]
)
```

---

### 1. Single URL Prediction

```python
@app.post("/api/v1/predict", response_model=PredictionResponse)
async def predict_url(request: PredictionRequest):
    """
    Predict if a URL is phishing or safe.

    Request:
    {
        "url": "https://example.com",
        "include_explanation": true,
        "fetch_content": false
    }

    Response:
    {
        "url": "https://example.com",
        "prediction": "phishing",
        "confidence": 0.87,
        "risk_score": 0.87,
        "processing_time_ms": 234,

        "model_scores": {
            "random_forest": 0.92,
            "xgboost": 0.85,
            "tgis": 0.15,
            "ensemble": 0.87
        },

        "api_checks": {
            "safe_browsing": {
                "is_flagged": true,
```

```
                "threat_types": ["SOCIAL_ENGINEERING"]
            },
            "whois": {
                "domain_age_days": 15,
                "registrar": "Namecheap"
            }
        },

        "graph_analysis": {
            "trust_score": 0.15,
            "cluster_risk": "high",
            "suspicious_neighbors": 12
        },

        "top_features": [
            {"name": "domain_age_days", "value": 15, "importance": 0.23},
            {"name": "tld_suspicious", "value": 1, "importance": 0.18}
        ],

        "explanation": {
            "shap_values": {...},
            "reason": "Domain is very new (15 days old), uses suspicious TLD..."
        }
    }

    Status Codes:
    - 200: Success
    - 400: Invalid URL format
    - 429: Rate limit exceeded
    - 500: Internal server error
    """
    pass
```

---

2. Batch Prediction

```
@app.post("/api/v1/batch-predict", response_model=BatchPredictionResponse)
async def batch_predict(request: BatchPredictionRequest):
    """
    Predict multiple URLs at once.

    Request:
    {
        "urls": [
            "https://example1.com",
            "https://example2.com",
            ...
        ],
        "max_urls": 100
    }
```

```
    Response:
    {
        "total_urls": 50,
        "processed": 50,
        "failed": 0,
        "phishing_count": 12,
        "safe_count": 38,
        "processing_time_ms": 5432,

        "results": [
            {
                "url": "https://example1.com",
                "prediction": "phishing",
                "confidence": 0.87,
                "risk_score": 0.87
            },
            ...
        ]
    }

    Limit: 100 URLs per request
    """
    pass
```

---

### 3. Detailed Analysis

```
@app.get("/api/v1/analyze/{analysis_id}", response_model=AnalysisResponse)
async def get_analysis(analysis_id: str):
    """
    Get detailed analysis report.

    Response:
    {
        "analysis_id": "abc123",
        "url": "https://example.com",
        "timestamp": "2024-01-15T10:30:00Z",
        "prediction": "phishing",

        "features": {
            "url_features": {...},
            "domain_features": {...},
            "content_features": {...},
            "graph_features": {...}
        },

        "model_details": {
            "rf_feature_importance": [...],
            "xgb_feature_importance": [...],
            "decision_path": [...]
        },
```

```
        "graph_visualization": {
            "nodes": [...],
            "edges": [...],
            "layout": "force-directed"
        }
    }
    """
    pass
```

---

4. Health Check

```
@app.get("/health")
async def health_check():
    """
    System health status.

    Response:
    {
        "status": "healthy",
        "version": "1.0.0",
        "uptime_seconds": 12345,

        "components": {
            "models": {
                "random_forest": "loaded",
                "xgboost": "loaded",
                "graph": "loaded"
            },
            "external_apis": {
                "safe_browsing": "reachable",
                "whois": "reachable"
            },
            "cache": "connected"
        },

        "metrics": {
            "total_predictions": 1234,
            "avg_response_time_ms": 245,
            "cache_hit_rate": 0.76
        }
    }
    """
    pass
```

---

5. Model Metrics

```python
@app.get("/api/v1/metrics")
async def get_metrics():
    """
    Model performance metrics.

    Response:
    {
        "test_set_performance": {
            "accuracy": 0.96,
            "precision": 0.94,
            "recall": 0.97,
            "f1_score": 0.95,
            "auc_roc": 0.98
        },

        "confusion_matrix": {
            "true_positive": 485,
            "true_negative": 492,
            "false_positive": 8,
            "false_negative": 15
        },

        "model_versions": {
            "random_forest": "v1.2.0",
            "xgboost": "v1.2.0",
            "last_trained": "2024-01-10"
        }
    }
    """
    pass
```

---

## 💾 Database Schema

### PostgreSQL Tables

```cpp
-- Predictions table
CREATE TABLE predictions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    url TEXT NOT NULL,
    prediction VARCHAR(20) NOT NULL,
    confidence FLOAT NOT NULL,
    risk_score FLOAT NOT NULL,
    rf_score FLOAT,
    xgb_score FLOAT,
    tgis_score FLOAT,
    processing_time_ms INTEGER,
    created_at TIMESTAMP DEFAULT NOW(),
    user_id UUID,
```

```sql
    INDEX idx_url_hash (MD5(url)),
    INDEX idx_created_at (created_at),
    INDEX idx_prediction (prediction)
);

-- Features table
CREATE TABLE features (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    prediction_id UUID REFERENCES predictions(id),
    features JSONB NOT NULL,  -- All 70 features
    created_at TIMESTAMP DEFAULT NOW()
);

-- External API responses
CREATE TABLE api_responses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    url TEXT NOT NULL,
    api_source VARCHAR(50) NOT NULL,
    response JSONB NOT NULL,
    cached_until TIMESTAMP NOT NULL,
    created_at TIMESTAMP DEFAULT NOW(),

    INDEX idx_url_source (MD5(url), api_source)
);

-- Trust graph nodes
CREATE TABLE graph_nodes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    node_type VARCHAR(20) NOT NULL,
    node_id TEXT NOT NULL,
    attributes JSONB,
    trust_score FLOAT,
    updated_at TIMESTAMP DEFAULT NOW(),

    UNIQUE (node_type, node_id)
);

-- Trust graph edges
CREATE TABLE graph_edges (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_node_id UUID REFERENCES graph_nodes(id),
    target_node_id UUID REFERENCES graph_nodes(id),
    edge_type VARCHAR(50) NOT NULL,
    weight FLOAT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Model performance logs
CREATE TABLE model_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_name VARCHAR(50) NOT NULL,
    version VARCHAR(20) NOT NULL,
    metric_name VARCHAR(50) NOT NULL,
    metric_value FLOAT NOT NULL,
```

```
    logged_at TIMESTAMP DEFAULT NOW()
);
```

---

## 🚀 Deployment Architecture

### Docker Compose Setup

**File:** `docker-compose.yml`

```yaml
version: "3.8"

services:
  # Main API service
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@postgres:5432/phishing_db
      - REDIS_URL=redis://redis:6379/0
      - SAFE_BROWSING_API_KEY=${SAFE_BROWSING_API_KEY}
    depends_on:
      - postgres
      - redis
    volumes:
      - ./data:/app/data
      - ./models:/app/models
    command: uvicorn api.main:app --host 0.0.0.0 --port 8000

  # Dashboard service
  dashboard:
    build: .
    ports:
      - "8501:8501"
    environment:
      - API_URL=http://api:8000
    depends_on:
      - api
    command: streamlit run dashboard/app.py

  # PostgreSQL database
  postgres:
    image: postgres:15
    environment:
      - POSTGRES_USER=phishing_user
      - POSTGRES_PASSWORD=secure_password
      - POSTGRES_DB=phishing_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
```

```yaml
      - "5432:5432"

  # Redis cache
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  # (Optional) Prometheus monitoring
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./config/prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus

  # (Optional) Grafana dashboard
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus

volumes:
  postgres_data:
  redis_data:
  prometheus_data:
  grafana_data:
```

---

### Dockerfile

```dockerfile
FROM python:3.10-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    curl \
    git \
    && rm -rf /var/lib/apt/lists/*
```

```
# Copy requirements
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create necessary directories
RUN mkdir -p data/models data/cache logs

# Expose ports
EXPOSE 8000 8501

# Default command (can be overridden in docker-compose)
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

---

## 📋 Implementation Checklist

### Phase 1: Foundation (Week 1)

```
Data Collection & Preprocessing
├── [ ] Download phishing datasets (PhishTank, OpenPhish)
├── [ ] Download legitimate URL dataset (Alexa, Tranco)
├── [ ] Create data loader scripts
├── [ ] Implement data cleaning pipeline
├── [ ] Split data (train/val/test: 70/15/15)
└── [ ] Document data sources and statistics

Project Setup
├── [ ] Initialize Git repository
├── [ ] Create directory structure
├── [ ] Setup virtual environment
├── [ ] Install core dependencies
├── [ ] Configure logging
├── [ ] Create .env.example file
└── [ ] Write initial README.md
```

### Phase 2: Feature Engineering (Week 2)

```
URL Features
├── [ ] Implement URLFeatureExtractor class
├── [ ] Add 15 URL structure features
├── [ ] Write unit tests
└── [ ] Validate on sample data

Domain Features
├── [ ] Setup WHOIS client with caching
├── [ ] Implement DNS resolver
```

```
├── [ ] Implement SSL checker
├── [ ] Add domain analysis features (20 total)
├── [ ] Handle API rate limits
└── [ ] Write integration tests

Content Features
├── [ ] Implement web scraper with timeout
├── [ ] Add HTML parsing logic
├── [ ] Extract 15 content features
├── [ ] Handle inaccessible pages gracefully
└── [ ] Test on various page types

Feature Pipeline
├── [ ] Create FeaturePipeline orchestrator
├── [ ] Implement parallel feature extraction
├── [ ] Add progress tracking
├── [ ] Save features to Parquet
└── [ ] Profile performance
```

### Phase 3: TGIS Implementation (Week 3)

```
Graph Construction
├── [ ] Design graph schema (nodes & edges)
├── [ ] Implement GraphBuilder class
├── [ ] Build graph from training data
├── [ ] Persist graph to disk (gpickle)
└── [ ] Visualize sample subgraphs

Trust Scoring
├── [ ] Implement trust propagation algorithm
├── [ ] Initialize known node scores
├── [ ] Run iterative updates
├── [ ] Validate convergence
└── [ ] Benchmark performance

Anomaly Detection
├── [ ] Extract graph metrics (centrality, clustering)
├── [ ] Implement community detection
├── [ ] Train Isolation Forest on graph features
├── [ ] Calculate anomaly scores
└── [ ] Evaluate on labeled data

Graph Features
├── [ ] Implement GraphFeatureExtractor
├── [ ] Add 10 graph-based features
├── [ ] Test on new URLs
└── [ ] Integrate with main pipeline
```

### Phase 4: ML Models (Week 4)

```
Data Preparation
├── [ ] Merge all features (URL + Domain + Content + Graph)
├── [ ] Handle missing values
├── [ ] Apply SMOTE for class balance
├── [ ] Normalize/scale features
└── [ ] Create feature importance baseline

Random Forest
├── [ ] Implement RandomForestTrainer
├── [ ] Setup hyperparameter grid
├── [ ] Run cross-validation
├── [ ] Select best model
├── [ ] Analyze feature importance
└── [ ] Save model artifact

XGBoost
├── [ ] Implement XGBoostTrainer
├── [ ] Setup early stopping
├── [ ] Tune hyperparameters
├── [ ] Compare with Random Forest
├── [ ] Generate SHAP values
└── [ ] Save model artifact

Ensemble
├── [ ] Implement EnsemblePredictor
├── [ ] Test weight combinations
├── [ ] Optimize ensemble weights
├── [ ] Validate on test set
└── [ ] Document performance metrics

Evaluation
├── [ ] Calculate accuracy, precision, recall, F1
├── [ ] Generate confusion matrix
├── [ ] Plot ROC curve and calculate AUC
├── [ ] Analyze errors (false positives/negatives)
└── [ ] Create evaluation report
```

### Phase 5: External APIs (Week 5)

```
Google Safe Browsing
├── [ ] Get API key from Google Cloud Console
├── [ ] Implement SafeBrowsingClient
├── [ ] Add request/response validation
├── [ ] Implement caching layer
├── [ ] Handle rate limits
└── [ ] Test with known malicious URLs

WHOIS Integration
├── [ ] Implement WHOISClient
├── [ ] Parse WHOIS responses
```

```
  ├── [ ] Extract domain metadata
  ├── [ ] Cache results (7 days)
  ├── [ ] Handle parsing errors
  └── [ ] Test across TLDs (.com, .net, .org, etc.)

  Caching System
  ├── [ ] Setup Redis container
  ├── [ ] Implement CacheManager
  ├── [ ] Define TTL policies
  ├── [ ] Add cache hit/miss metrics
  └── [ ] Test cache invalidation
```

### Phase 6: REST API (Week 6)

```
  API Development
  ├── [ ] Initialize FastAPI project
  ├── [ ] Define Pydantic schemas (request/response)
  ├── [ ] Implement /predict endpoint
  ├── [ ] Implement /batch-predict endpoint
  ├── [ ] Implement /analyze/{id} endpoint
  ├── [ ] Implement /health endpoint
  ├── [ ] Implement /metrics endpoint
  ├── [ ] Add input validation
  ├── [ ] Add error handling
  └── [ ] Add request logging

  Middleware & Security
  ├── [ ] Add CORS middleware
  ├── [ ] Implement rate limiting
  ├── [ ] Add API key authentication (optional)
  ├── [ ] Add request/response logging
  └── [ ] Add security headers

  Testing
  ├── [ ] Write unit tests for endpoints
  ├── [ ] Write integration tests
  ├── [ ] Test error scenarios
  ├── [ ] Load test with locust/artillery
  └── [ ] Document API with OpenAPI/Swagger
```

### Phase 7: Dashboard (Week 7)

```
  Streamlit Dashboard
  ├── [ ] Create main app layout
  ├── [ ] Add URL input form
  ├── [ ] Display prediction results
  ├── [ ] Visualize feature importance
  ├── [ ] Show graph visualization
  ├── [ ] Add batch upload functionality
```

```
├── [ ] Display performance metrics
├── [ ] Add export to CSV functionality
└── [ ] Style with custom CSS

Interactive Components
├── [ ] Real-time prediction updates
├── [ ] Feature contribution chart
├── [ ] Trust graph network diagram
├── [ ] Historical analysis charts
└── [ ] Model comparison view
```

### Phase 8: Database & Persistence (Week 8)

```
PostgreSQL Setup
├── [ ] Create database schema
├── [ ] Implement database models (SQLAlchemy)
├── [ ] Create migrations (Alembic)
├── [ ] Setup connection pooling
└── [ ] Add indexes for performance

Data Persistence
├── [ ] Save predictions to database
├── [ ] Store features for each prediction
├── [ ] Cache API responses in DB
├── [ ] Store graph data
└── [ ] Implement cleanup jobs (old data)
```

### Phase 9: Deployment (Week 9)

```
Docker Setup
├── [ ] Create Dockerfile
├── [ ] Create docker-compose.yml
├── [ ] Test local deployment
├── [ ] Optimize image size
└── [ ] Setup health checks

Environment Configuration
├── [ ] Create .env.example
├── [ ] Document all environment variables
├── [ ] Setup secrets management
└── [ ] Configure for dev/staging/prod

CI/CD (Optional)
├── [ ] Setup GitHub Actions / GitLab CI
├── [ ] Add automated testing
├── [ ] Add linting (flake8, black, mypy)
├── [ ] Add security scanning
└── [ ] Automate deployments
```

### Phase 10: Monitoring & Optimization (Week 10)

```
Monitoring
├── [ ] Setup Prometheus metrics
├── [ ] Create Grafana dashboards
├── [ ] Add custom metrics (prediction latency, etc.)
├── [ ] Setup alerting rules
└── [ ] Monitor API error rates

Performance Optimization
├── [ ] Profile feature extraction speed
├── [ ] Optimize database queries
├── [ ] Implement batch processing
├── [ ] Add connection pooling
└── [ ] Optimize model inference

Documentation
├── [ ] Write comprehensive README
├── [ ] Document API endpoints
├── [ ] Create architecture diagram
├── [ ] Write TGIS algorithm explanation
├── [ ] Add usage examples
└── [ ] Create deployment guide
```

---

## 🎯 Success Metrics

### Model Performance Targets

```
Accuracy:     ≥ 95%
Precision:    ≥ 94% (minimize false positives)
Recall:       ≥ 96% (minimize false negatives)
F1 Score:     ≥ 95%
AUC-ROC:      ≥ 0.98
```

### API Performance Targets

```
Average Response Time:  < 300ms
P95 Response Time:      < 500ms
P99 Response Time:      < 1000ms
Uptime:                 ≥ 99.5%
Cache Hit Rate:         ≥ 70%
```

### TGIS Contribution

```
Improvement over baseline ML:  ≥ 2% accuracy
False positive reduction:      ≥ 15%
Novel phishing detection:      ≥ 80% of zero-day phishing
```

---

## Key Technologies Summary

| Category | Technologies |
|---|---|
| **Core ML** | scikit-learn, XGBoost, SHAP |
| **Graph** | NetworkX, python-igraph, Louvain |
| **Web** | FastAPI, Uvicorn, Streamlit |
| **Data** | Pandas, NumPy, Parquet |
| **Database** | PostgreSQL, Redis, SQLAlchemy |
| **External APIs** | Google Safe Browsing, python-whois, dnspython |
| **Deployment** | Docker, Docker Compose |
| **Testing** | pytest, locust |
| **Monitoring** | Prometheus, Grafana (optional) |

---

## Environment Variables

**File:** `.env.example`

```
# API Keys
SAFE_BROWSING_API_KEY=your_google_api_key_here

# Database
DATABASE_URL=postgresql://user:password@localhost:5432/phishing_db
REDIS_URL=redis://localhost:6379/0

# API Configuration
API_HOST=0.0.0.0
API_PORT=8000
API_WORKERS=4

# Model Paths
MODEL_DIR=/app/data/models
GRAPH_DIR=/app/data/graphs

# Feature Flags
ENABLE_CONTENT_FEATURES=true
ENABLE_GRAPH_FEATURES=true
ENABLE_SAFE_BROWSING=true
```

```
# Rate Limiting
RATE_LIMIT_PER_MINUTE=60
BATCH_MAX_URLS=100

# Caching
CACHE_SAFE_BROWSING_HOURS=24
CACHE_WHOIS_HOURS=168
CACHE_DNS_HOURS=1

# Logging
LOG_LEVEL=INFO
LOG_FILE=/app/logs/app.log
```

---

## 🎓 What This Architecture Demonstrates

### Technical Skills

☑ **Advanced ML Engineering**: Feature engineering, ensemble methods, handling imbalanced data ☑ **Graph Theory Application**: Novel TGIS approach, trust propagation, community detection ☑ **API Integration**: Google Safe Browsing, WHOIS, DNS, SSL validation ☑ **Production System Design**: Caching, rate limiting, error handling, monitoring ☑ **Full-Stack Development**: Backend API, frontend dashboard, database design ☑ **DevOps**: Docker containerization, CI/CD, deployment

### Research Mindset

☑ **Novel Approach**: TGIS is a unique contribution combining graph theory with ML ☑ **Multi-Modal Analysis**: URL structure + domain metadata + content + graph ☑ **Explainability**: SHAP values provide interpretable predictions ☑ **Continuous Improvement**: Designed for active learning and retraining

### Security Expertise

☑ **Threat Understanding**: Deep knowledge of phishing techniques ☑ **Defense-in-Depth**: Multiple detection layers (ML + APIs + graph) ☑ **Real-World Validation**: Integration with industry-standard security APIs

---

## 🚀 Next Steps

1. **Start with Phase 1**: Setup project structure and download datasets
2. **Implement Features Incrementally**: Begin with URL features, then domain, then graph
3. **Test Each Component**: Write tests before moving to next phase
4. **Iterate on TGIS**: This is your differentiator - invest time here
5. **Document Everything**: Good documentation makes the project interview-ready

---

## 📝 Additional Resources

### Datasets

- PhishTank: https://www.phishtank.com/developer_info.php
- OpenPhish: https://openphish.com/
- Alexa Top Sites: https://www.alexa.com/topsites
- Tranco List: https://tranco-list.eu/

### APIs

- Google Safe Browsing: https://developers.google.com/safe-browsing
- WHOIS: https://pypi.org/project/python-whois/

### Papers (for TGIS inspiration)

- "Graph-based Phishing Detection" (various IEEE papers)
- "Trust Propagation in Social Networks"
- "Anomaly Detection in Dynamic Networks"

---

**This architecture is designed to be implemented iteratively with AI assistance. Use this document as a reference guide and implement each component step-by-step.**

**Good luck building your elite phishing detection system! 🌓 🚀**